KEY TERM

Nested loop: loop containing another loop

12.08 Stepwise refinement

Many problems that we want to solve are bigger than the ones we met so far. To make it easier to solve a bigger problem, we break the problem down into smaller steps. These might need breaking down further until the steps are small enough to solve easily.

For a solution to a problem to be programmable, we need to break down the steps of the solution into the basic constructs of sequence, assignment, selection, repetition, input and output.

We can use a method called **stepwise refinement** to break down the steps of our outline solution into smaller steps until it is detailed enough. In Section 12.02 we looked at a recipe for a cake. The step of mixing together all the ingredients was broken down into more detailed steps.

KEY TERM

Stepwise refinement: breaking down the steps of an outline solution into smaller and smaller steps

WORKED EXAMPLE 12.10

Drawing a pyramid using stepwise refinement

The problem to be solved: Take as input a chosen symbol and an odd number. Output a pyramid shape made up entirely of the chosen symbol, with the number of symbols in the final row matching the number input.

For example the two input values A and 9 result in the following output:

```
    A
   AAA
  AAAAA
 AAAAAAA
AAAAAAAAA
```

This problem is similar to Worked Example 12.09, but the number of symbols in each row starts with one and increases by two with each row. Each row starts with a decreasing number of spaces, to create the slope effect.

Our first attempt at solving this problem using structured English is:

```
01   Set up initial values
02   REPEAT
03       Output number of spaces
04       Output number of symbols
05       Adjust number of spaces and number of symbols to be output in next row
06   UNTIL the required number of symbols have been output in one row
```

The steps are numbered to make it easier to refer to them later.

This is not enough detail to write a program in a high-level programming language. Exactly what values do we need to set?

(Continued)

We need as input:

- the symbol character from which the pyramid is to be formed
- the number of symbols in the final row (for the pyramid to look symmetrical, this needs to be an odd number).

We need to calculate how many spaces we need in the first row. So that the slope of the pyramid is symmetrical, this number should be half of the final row's symbols. We need to set the number of symbols to be output in the first row to 1. We therefore need the identifiers listed in Table 12.12.

Identifier	Explanation
`Symbol`	The character symbol to form the pyramid
`MaxNumberOfSymbols`	The number of symbols in the final row
`NumberOfSpaces`	The number of spaces to be output in the current row
`NumberOfSymbols`	The number of symbols to be output in the current row

Table 12.12 Identifier table for pyramid example

Using pseudocode, we now refine the steps of our first attempt. To show which step we are refining, a numbering system is used as shown.

Step 01 can be broken down as follows:

```
01    // Set up initial values expands into:
01.1 INPUT Symbol
01.2 INPUT MaxNumberOfSymbols
01.3 NumberOfSpaces ← (MaxNumberOfSymbols - 1) / 2
01.4 NumberOfSymbols ← 1
```


Remember we need an odd number for `MaxNumberOfSymbols`. We need to make sure the input is an odd number. So we further refine Step 01.2:

```
01.2    // INPUT MaxNumberOfSymbols expands into:
01.2.1 REPEAT
01.2.2     INPUT MaxNumberOfSymbols
01.2.3 UNTIL MaxNumberOfSymbols MOD 2 = 1
01.2.4 // MOD 2 gives the remainder after integer division by 2
```

We can now look to refine Steps 03 and 04:

```
03 // Output number of spaces expands into:
03.1   FOR i ← 1 TO NumberOfSpaces
03.2       OUTPUT Space // without moving to next line
03.3   NEXT i

04     // Output number of symbols expands into:
04.1   FOR i ← 1 TO NumberOfSymbols
04.2       OUTPUT Symbol // without moving to next line
04.3   NEXT i
04.4   OUTPUT Newline // move to the next line
```

In Step 05 we need to decrease the number of spaces by 1 and increase the number of symbols by 2:

```
05    // Adjust values for next row expands into:
05.1 NumberOfSpaces ← NumberOfSpaces - 1
05.2 NumberOfSymbols ← NumberOfSymbols + 2
```

(Continued)

Step 06 essentially checks whether the number of symbols for the next row is now greater than the value input at the beginning.

```
06      UNTIL NumberOfSymbols > MaxNumberOfSymbols
```

We can put together all the steps and end up with a solution.

```
01      // Set Values
01.1    INPUT Symbol
01.2    // Input max number of symbols (an odd number)
01.2.1  REPEAT
01.2.2      INPUT MaxNumberOfSymbols
01.2.3  UNTIL MaxNumberOfSymbols MOD 2 = 1
01.3    NumberOfSpaces ← (MaxNumberOfSymbols - 1) / 2
01.4    NumberOfSymbols ← 1
02      REPEAT
03          // Output number of spaces
03.1        FOR i ← 1 TO NumberOfSpaces
03.2            OUTPUT Space // without moving to next line
03.3        NEXT i
04          // Output number of symbols
04.1        FOR i ← 1 TO NumberOfSymbols
04.2            OUTPUT Symbol // without moving to next line
04.3        NEXT i
04.4        OUTPUT Newline // move to the next line
05          // Adjust Values For Next Row
05.1        NumberOfSpaces ← NumberOfSpaces - 1
05.2        NumberOfSymbols ← NumberOfSymbols + 2
06      UNTIL NumberOfSymbols > MaxNumberOfSymbols
```


TASK 12.06

Use stepwise refinement to output a hollow triangle. For example the two input values A and 9 result in the following output:

```
    A
   A A
  A   A
 A     A
AAAAAAAAA
```

A first attempt at solving this problem using structured English is:

```
01   Set up initial values
02   REPEAT
03       Output leading number of spaces
04       Output symbol, middle spaces, symbol
05       Adjust number of spaces and number of symbols to be output in
         next row
06   UNTIL the required number of symbols have been output in one row
```

12.09 Modules

Another method of developing a solution is to decompose the problem into sub-tasks. Each sub-task can be considered as a 'module' that is refined separately. Modules are procedures and functions.

A **procedure** groups together a number of steps and gives them a name (an identifier). We can use this identifier when we want to refer to this group of steps. When we want to perform the steps in a procedure we call the procedure by its name.

KEY TERM

Procedure: a sequence of steps that is given an identifier and can be called to perform a sub-task

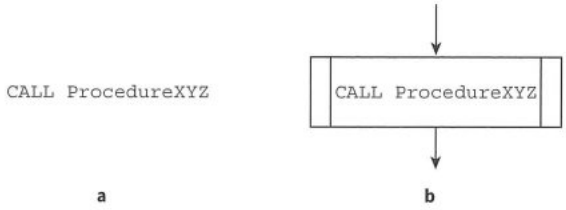


Figure 12.11 Representation of a procedure in (a) pseudocode and (b) a flowchart

A **function** groups together a number of steps and gives them a name (an identifier). These steps produce and return a value that is used in an expression. Worked Example 12.12 uses functions.

KEY TERM

Function: a sequence of steps that is given an identifier and returns a single value; function call is part of an expression

Note: Because a function returns a value, the function definition states the data type of this value. See more about data types in Chapter 13.

The rules for module identifiers are the same as for variable identifiers (see Section 12.04)

WORKED EXAMPLE 12.11

Drawing a pyramid using modules

The problem is the same as in Worked Example 12.10.

When we want to set up the initial values, we call a procedure, using the following statement:

```
CALL SetValues
```

We can rewrite the top-level solution to our pyramid problem using a procedure for each step, as:

```
CALL SetValues
REPEAT
    CALL OutputSpaces
    CALL OutputSymbols
    CALL AdjustValuesForNextRow
UNTIL NumberOfSymbols > MaxNumberOfSymbols
```

(Continued)

This top-level solution calls four procedures. This means each procedure has to be defined. The procedure definitions are:

```
PROCEDURE SetValues
    INPUT Symbol
    CALL InputMaxNumberOfSymbols // need to ensure it is an odd number
    NumberOfSpaces ← (MaxNumberOfSymbols - 1) / 2
    NumberOfSymbols ← 1
ENDPROCEDURE

PROCEDURE InputMaxNumberOfSymbols
    REPEAT
        INPUT MaxNumberOfSymbols
    UNTIL MaxNumberOfSymbols MOD 2 = 1
ENDPROCEDURE

PROCEDURE OutputSpaces
    FOR Count1 ← 1 TO NumberOfSpaces
        OUTPUT Space // without moving to next line
    NEXT Count1
ENDPROCEDURE

PROCEDURE OutputSymbols
    FOR Count2 ← 1 TO NumberOfSymbols
        OUTPUT Symbol // without moving to next line
    NEXT Count2
    OUTPUT Newline // move to the next line
ENDPROCEDURE

PROCEDURE AdjustValuesForNextRow
    NumberOfSpaces ← NumberOfSpaces - 1
    NumberOfSymbols ← NumberOfSymbols + 2
ENDPROCEDURE
```


TASK 12.07

Amend your algorithm for Task 12.05 to use modules.

WORKED EXAMPLE 12.12

Drawing a pyramid using modules

The problem is the same as in Worked Example 12.11.

We can rewrite the top-level solution to our pyramid problem using procedures and functions.

```
01    CALL SetValues
02    REPEAT
03        CALL OutputSpaces
04        CALL OutputSymbols
05.1      NumberOfSpaces ← AdjustedNumberOfSpaces
05.2      NumberOfSymbols ← AdjustedNumbeOfSymbols
06    UNTIL NumberOfSymbols > MaxNumberOfSymbols
```

(Continued)

This top-level solution calls three procedures. It also makes use of two functions in lines 05.1 and 05.2.

The procedures and functions have to be defined.

```
PROCEDURE SetValues
    INPUT Symbol
    MaxNumberOfSymbols ← ValidatedMaxNumberOfSymbols
    NumberOfSpaces ← (MaxNumberOfSymbols - 1) / 2
    NumberOfSymbols ← 1
ENDPROCEDURE

FUNCTION ValidatedMaxNumberOfSymbols RETURNS INTEGER
    REPEAT
        INPUT MaxNumberOfSymbols
    UNTIL MaxNumberOfSymbols MOD 2 = 1
    RETURN MaxNumberOfSymbols
ENDFUNCTION

PROCEDURE OutputSpaces
    FOR Count1 ← 1 TO NumberOfSpaces
        OUTPUT Space // without moving to next line
    NEXT Count1
ENDPROCEDURE

PROCEDURE OutputSymbols
    FOR Count2 ← 1 TO NumberOfSymbols
       OUTPUT Symbol // without moving to next line
    NEXT Count2
    OUTPUT Newline // move to the next line
ENDPROCEDURE

FUNCTION AdjustedNumberOfSpaces RETURNS INTEGER
    NumberOfSpaces ← NumberOfSpaces - 1
    RETURN NumberOfSpaces
ENDFUNCTION

FUNCTION AdjustedNumberOfSymbols RETURNS INTEGER
    NumberOfSymbols ← NumberOfSymbols + 2
    RETURN NumberOfSymbols
ENDFUNCTION
```

Note that procedure `SetValues` uses a function `ValidatedMaxNumberOfSymbols`.

One benefit of using modules is that individual modules can be reused in other solutions. Therefore, modules should be designed to be self-contained. That means they should not rely on external variables. All variables that are required by a module should be passed to it using parameters. To illustrate this, look at Worked Example 12.13

WORKED EXAMPLE 12.13

Drawing a pyramid using modules and parameters

The problem is the same as in Worked Example 12.12.

```
01    CALL SetValues(Symbol, MaxNumberOfSymbols, NumberOfSpaces, NumberOfSymbols)
02    REPEAT
03        CALL OutputSpaces(NumberOfSpaces)
04        CALL OutputSymbols(NumberOfSymbols, Symbol)
05.1      NumberOfSpaces ← AdjustedNumberOfSpaces(NumberOfSpaces)
05.2      NumberOfSymbols ← AdjustedNumbeOfSymbols(NumberOfSymbols)
06    UNTIL NumberOfSymbols > MaxNumberOfSymbols
```

Module definitions:

```
PROCEDURE SetValues(Symbol, MaxNumberOfSymbols, NumberOfSpaces, NumberOfSymbols)
    INPUT Symbol
    MaxNumberOfSymbols ← ValidatedMaxNumberOfSymbols
    NumberOfSpaces ← (MaxNumberOfSymbols - 1) / 2
    NumberOfSymbols ← 1
ENDPROCEDURE

FUNCTION ValidatedMaxNumberOfSymbols RETURNS INTEGER
    REPEAT
        INPUT MaxNumberOfSymbols
    UNTIL MaxNumberOfSymbols MOD 2 = 1
    RETURN MaxNumberOfSymbols
ENDFUNCTION

PROCEDURE OutputSpaces(NumberOfSpaces)
    FOR Count1 ← 1 TO NumberOfSpaces
        OUTPUT Space // without moving to next line
    NEXT Count1
ENDPROCEDURE

PROCEDURE OutputSymbols(NumberOfSymbols, Symbol)
    FOR Count2 ← 1 TO NumberOfSymbols
        OUTPUT Symbol // without moving to next line
    NEXT Count2
    OUTPUT Newline // move to the next line
ENDPROCEDURE

FUNCTION AdjustedNumberOfSpaces(NumberOfSpaces) RETURNS INTEGER
    NumberOfSpaces ← NumberOfSpaces - 1
    RETURN NumberOfSpaces
ENDFUNCTION

FUNCTION AdjustedNumbeOfSymbols(NumberOfSymbols) RETURNS INTEGER
    NumberOfSymbols ← NumberOfSymbols + 2
    RETURN NumberOfSymbols
ENDFUNCTION
```

Note that the procedure `OutputSpaces` uses a variable, `Count1`, which is used only within the module. Similarly, `OutputSymbols` uses variable `Count2` only within the module. We call such a variable a **local variable** (see Chapter 14, Section 14.09). A variable available to all modules is known as a **global variable** (see Chapter 14, Section 14.09).

TIP

Good design uses local variables as it makes modules independent and re-usable.

KEY TERMS

Local variable: a variable that is accessible only within the module in which it is declared

Global variable: a variable that is accessible from all modules

Reflection Point:

Can you think of other problems and use decomposition to break them down into basic constructs, input and output statements?

Summary

- Abstraction involves filtering out information that is not needed to solve the problem.
- Decomposition is breaking down problems into sub-problems, leading to the concept of a program module.
- An algorithm is a sequence of steps that can be carried out to solve a problem.
- Algorithms are expressed using the four basic constructs of assignment, sequence, selection and repetition.
- Algorithms can be documented using pseudocode.
- Stepwise refinement: breaking down the steps of an outline solution into smaller and smaller steps.
- Logic statements use the relational operators =, <, >, <>, <= and >= and the logic operators AND, OR and NOT.
- Selection constructs and conditional loops use conditions to determine the steps to be followed.

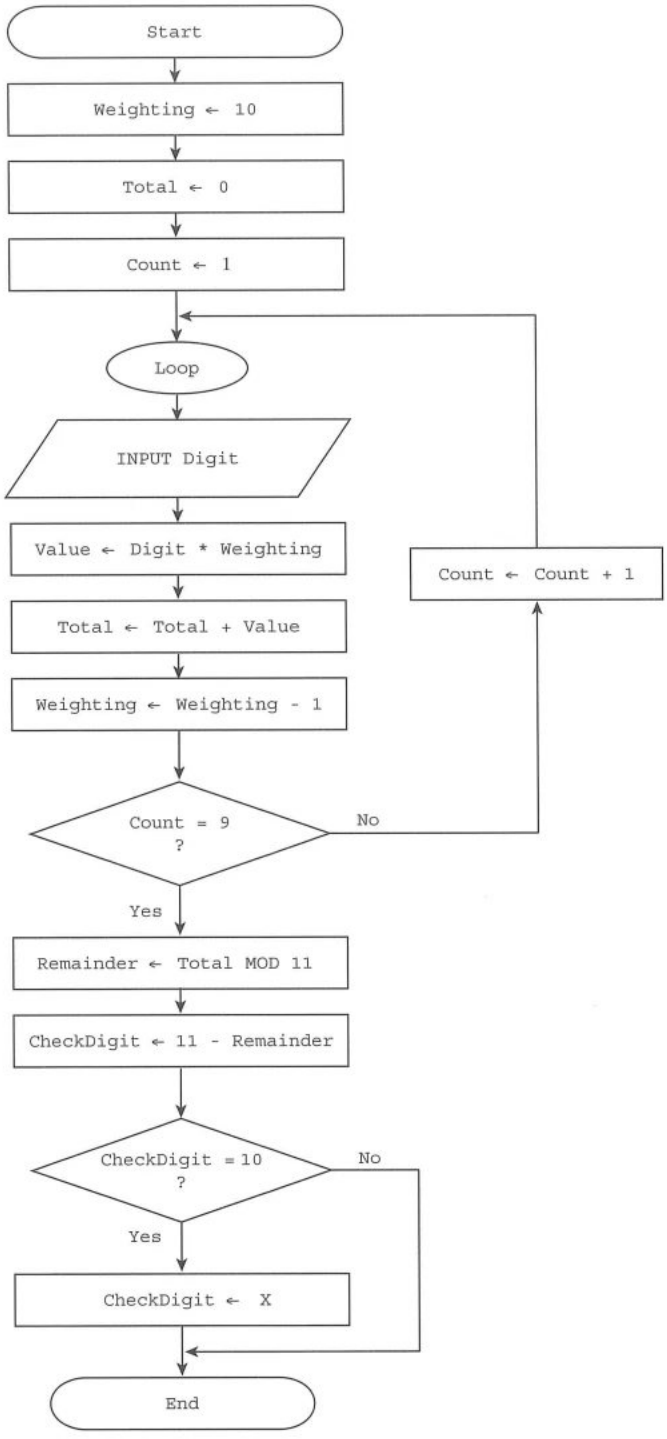
Exam-style Questions

1 The Modulo-11 method of calculating a check digit for a sequence of nine digits is as follows:

Each digit in the sequence is given a weight depending on its position in the sequence. The leftmost digit has a weight of 10. The next digit to the right has a weight of 9, the next one 8 and so on. Values are calculated by multiplying each digit by its weight. These values are added together and the sum is divided by 11. The remainder from this division is subtracted from 11 and this value is the check digit. If this value is 10, then the check digit is X. Note that `x MOD y` gives the remainder from the division of `x` by `y`.

The flowchart shows the algorithm for calculating the Modulo-11 check digit.

Write pseudocode from the flowchart.


Start
Weighting ← 10
Total ← 0
Count ← 1
Loop
INPUT Digit
Value ← Digit * Weighting
Total ← Total + Value
Weighting ← Weighting - 1
Count = 9 ?
No
Count ← Count + 1
Yes
Remainder ← Total MOD 11
CheckDigit ← 11 - Remainder
CheckDigit = 10 ?
No
Yes
CheckDigit ← X
End

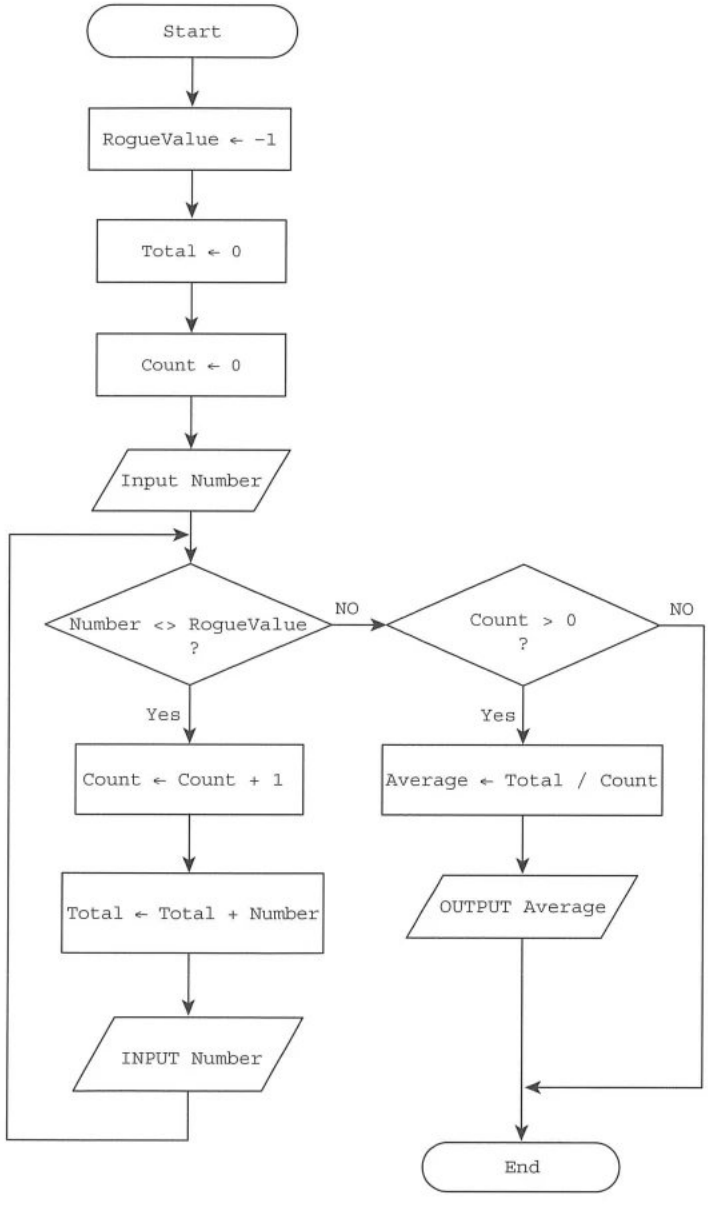
[9]

2 Write pseudocode for the following problem given in structured English.

```
REPEAT the following UNTIL the number input is zero
INPUT a number
Check whether number is positive or negative
Increment positive number count if the number is positive
Increment negative number count if the number is negative
```

[7]

3 Write pseudocode from the given flowchart. Use a `WHILE` loop.



[8]